

Data Processing

Introduction to ARM Cortex-M Microprocessors — Chapter 6

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

1	Why This Matters	
2	Arithmetic	
3	Logic and Bit Manipulation	
4	Shift Basics	
5	The Barrel Shifter	
6	Addressing Modes	
7	Dynamic Indexed Modes	
8	Design Challenge	

1	Why This Matters	
2	Arithmetic	
3	Logic and Bit Manipulation	
4	Shift Basics	
5	The Barrel Shifter	
6	Addressing Modes	
7	Dynamic Indexed Modes	
8	Design Challenge	

Last chapter showed how data moves between memory and registers with LDR and STR. This chapter asks the next question: once data is in a register, what can ARM do with it? Arithmetic changes values; logical instructions change individual bits; and a hardware barrel shifter lets ARM fold a shift into almost any instruction, including the ones that calculate an address. After this chapter, when you write `GPIOA->MODER |= (1U << 10)`, you will know exactly which instructions the processor runs, and why each one is needed.

After completing this chapter, you should be able to:

1. Choose the correct arithmetic instruction, and explain why a 32-bit constant may need more than one instruction.
2. Write a read-modify-write sequence that changes a bit field without harming adjacent bits.
3. Predict the result of a shift instruction, and distinguish logical from arithmetic shifts.
4. Use the flexible second operand, including barrel-shifter forms, to combine a shift with an arithmetic operation.
5. Use offset addressing, pre-indexed, and post-indexed forms to reach a structure member, and state which registers each mode changes.
6. Use a scaled register offset to reach an array element, and trace a complete array-processing program.

- 1 Why This Matters
- 2 **Arithmetic**
- 3 Logic and Bit Manipulation
- 4 Shift Basics
- 5 The Barrel Shifter
- 6 Addressing Modes
- 7 Dynamic Indexed Modes
- 8 Design Challenge

Instruction	Operation	Note
ADD	$Rd = Rn + \text{Operand2}$	
SUB	$Rd = Rn - \text{Operand2}$	
RSB	$Rd = \text{Operand2} - Rn$	Negation: RSB R0, R1, #0
MUL	$Rd = Rn \times Rm$	Single cycle on the M4
SDIV / UDIV	$Rd = Rn \div Rm$	2–12 cycles

Loading a 32-bit constant

The immediate value must fit *inside* the instruction, which is itself only 32 bits. So a full constant needs either **MOVW** then **MOVT** (two instructions, no memory access), or **LDR Rn, =value** (one instruction plus a load from a literal pool).

In C

```
total = a + b - c;
```

In assembly

```
LDR R0, =a
LDR R0, [R0] ; R0 = a
LDR R1, =b
LDR R1, [R1] ; R1 = b
LDR R2, =c
LDR R2, [R2] ; R2 = c
ADD R3, R0, R1 ; R3 = a + b
SUB R3, R3, R2 ; R3 = (a+b) - c
LDR R4, =total
STR R3, [R4]
```

One C statement, one register-only computation

Every operand is **loaded** into a register before either ADD or SUB can touch it. Neither instruction ever reads or writes memory directly – the result is **stored** only once, at the very end.

The compiler is free to reorder or reuse registers, but the shape never changes: **load every operand, compute in registers, store the result.**

1	<u>Why This Matters</u>
2	<u>Arithmetic</u>
3	<u>Logic and Bit Manipulation</u>
4	<u>Shift Basics</u>
5	<u>The Barrel Shifter</u>
6	<u>Addressing Modes</u>
7	<u>Dynamic Indexed Modes</u>
8	<u>Design Challenge</u>

Instruction	Operation	What it is for
AND	$Rd = Rn \& Op2$	Masking – keep only bits you care about
ORR	$Rd = Rn Op2$	Setting bits to 1
EOR	$Rd = Rn \wedge Op2$	Toggling bits
BIC	$Rd = Rn \& \sim Op2$	Clearing bits to 0

BIC is the one without a matching C operator

In C you clear bits with $x \&= \sim \text{mask}$. ARM gives this its own instruction. So the compiler emits *one* BIC, where you might expect MVN followed by AND.

R0 contains 12₁₀, R1 contains 10₁₀.

	3	2	1	0	
R0	1	1	0	0	R0 = 1100
R1	1	0	1	0	R1 = 1010
AND	1	0	0	0	R2, R0, R1 — keep where both are 1
ORR	1	1	1	0	R2, R0, R1 — set where either is 1
EOR	0	1	1	0	R2, R0, R1 — set where they differ
BIC	0	1	0	0	R2, R0, R1 — clear where R1 is 1

Read-Modify-Write The sequence behind every peripheral write

In C

```
GPIOA->MODER &=
    ~(3U << 10);

GPIOA->MODER |=
    (1U << 10);
```

In assembly

```
LDR R3, =0x40020000
LDR R0, [R3]
BIC R0, R0, #0xC00    ; clear bits 11:10
ORR R0, R0, #0x400    ; set bit 10
STR R0, [R3]
```

Why you must keep the neighbouring bits

This register holds settings for **sixteen pins**. Writing the whole register would reconfigure every one of them. **Load it, change only your bits, then store it back.**

$3U \ll 10 = 0xC00$; $1U \ll 10 = 0x400$. This exact sequence sets up PA5 as an output.

Following the Bits Through Clear-Then-Set



ORR alone cannot turn 11 into 01 — OR can only turn bits **on**. BIC must clear the field first.

Step	Assembly	Equivalent C
Load the register	LDR R1, [R0]	implicit in GPIOA->MODER
Clear the field	BIC R1, R1, R2	$\&= \sim(3U \ll 10)$
Set the new value	ORR R1, R1, #0x400	$ = (1U \ll 10)$
Store the register	STR R1, [R0]	implicit in the -> assignment

Two rows are “implicit”

C never issues a separate load or store on its own – the compiler emits LDR/STR automatically around every ->MODER reference. Only the middle two rows have a direct C spelling.

- 1 Why This Matters
- 2 Arithmetic
- 3 Logic and Bit Manipulation
- 4 Shift Basics**
- 5 The Barrel Shifter
- 6 Addressing Modes
- 7 Dynamic Indexed Modes
- 8 Design Challenge

Shift operations move all bits in a register left or right by a specified number of positions, useful for fast arithmetic and bit manipulation.

Connection to Digital Electronics

You have already encountered these concepts in Digital Electronics as hardware shift registers (SISO, SIPO, PISO, PIPO) and ring counters.

Instruction	Meaning	Effect
LSL #n	Logical shift left	Multiplies by 2^n ; zeros fill in from the right
LSR #n	Logical shift right	Divides an unsigned value by 2^n ; zeros fill in from the left
ASR #n	Arithmetic shift right	Divides a signed value by 2^n ; the sign bit fills in from the left
ROR #n	Rotate right	Bits that fall off the right re-enter on the left

Only one bit decides LSR versus ASR

Both shift right. The difference is what fills the vacated bits: LSR always fills with **0**. ASR copies the **sign bit**, so a negative number stays negative after the shift.

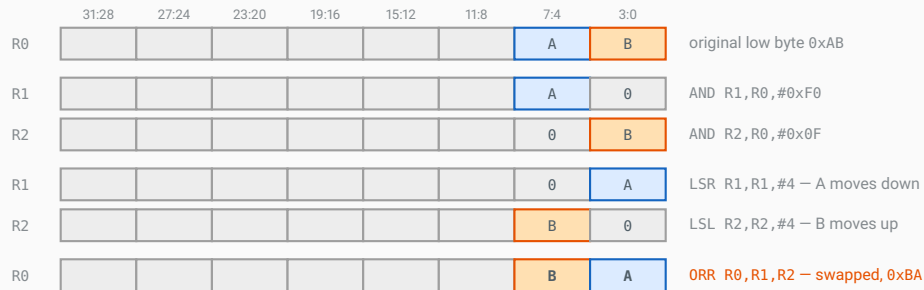
```
LSL R0, R1, #2    ; R0 = R1 << 2    (R1 times 4)
LSR R0, R1, #1    ; R0 = R1 >> 1    (R1 divided by 2, unsigned)
ASR R0, R1, #1    ; R0 = R1 >> 1    (R1 divided by 2, signed)
```

Written this way, a shift is a complete instruction: it reads R1, shifts it, and writes the result to R0 — exactly like ADD or MOV.

The idea worth carrying forward

A shift is useful by itself. But ARM can also feed a shifted value **directly into another instruction**, without a separate shift step first. That is the subject of the next section.

Swapping Two Nibbles Isolate with AND, recombine with ORR



Multiplying by a Power of Two Strength reduction: replacing multiply with shift

In C

```
total = 4 * a + b;
```

a and b are already in R0 and R1 — no LDR needed here.

In assembly

```
LSL  R2, R0, #2 ; R2 = a << 2  
      ;        = a * 4  
ADD  R3, R2, R1 ; R3 = (4*a) + b
```

Why the compiler prefers LSL over MUL

Multiplying by 4 is multiplying by 2^2 . A left shift by 2 bits produces exactly that result, and on most cores a shift is **cheaper** than a general multiply. The compiler makes this substitution automatically — it is called **strength reduction**. No MUL instruction appears at all.

This only works for a power-of-two constant. $4 * a$ becomes a shift; $5 * a$ still needs a real MUL.

1	Why This Matters
2	Arithmetic
3	Logic and Bit Manipulation
4	Shift Basics
5	The Barrel Shifter
6	Addressing Modes
7	Dynamic Indexed Modes
8	Design Challenge

A **barrel shifter** is a specialized digital circuit that can shift or rotate a data word by **any number of bit positions in a single clock cycle**.

Why is it a game-changer in ARM?

- **Hardware Efficiency:** Unlike a basic shift register (which shifts 1 bit per cycle), it uses combinational multiplexers to shift n bits instantly.
- **Zero-Cost Operations:** In ARM processors, it is wired directly into the ALU input path.
- **Combined Execution:** Shifts and arithmetic/logic operations happen in the **same instruction cycle** with zero clock penalty.

ADD R0, R1, R2, LSL #2 $\longrightarrow$ Shift R2 left by 2, then add to R1 in **1 cycle**.

The Flexible Second Operand

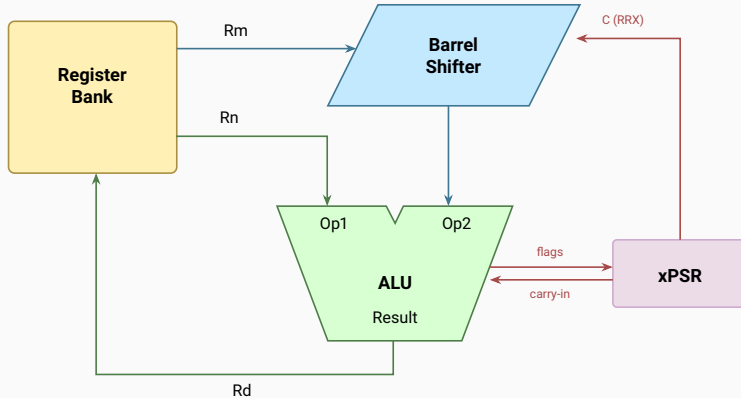
Opcode{S}{cond} Rd, Rn, Operand2

Form	Written	Meaning
Constant	#constant	An immediate encoded in the instruction
Register	Rm	A register holding the second operand
Register with shift	Rm, LSL #n	Rm shifted before use

```
ADD R0, R1, R2, LSR #3    ; R0 = R1 + (R2 >> 3)
ORR R0, R1, R2, LSL #2    ; R0 = R1 | (R2 << 2)
```

Folded into the instruction, not a separate step

The **barrel shifter** sits in the datapath *before* the ALU, so the shift and the operation happen together: no separate shift instruction is needed. Rm itself stays **unchanged** – only the value entering the ALU is shifted.



Rm feeds the barrel shifter. Its output enters the ALU as **Op2**. **Rn** feeds **Op1** directly. The result is written back to **Rd**, and the flags update in **xPSR**. The carry bit of **xPSR** is sent to ALU and barrel shifter.

1	<u>Why This Matters</u>
2	<u>Arithmetic</u>
3	<u>Logic and Bit Manipulation</u>
4	<u>Shift Basics</u>
5	<u>The Barrel Shifter</u>
6	<u>Addressing Modes</u>
7	<u>Dynamic Indexed Modes</u>
8	<u>Design Challenge</u>

The Four Addressing Modes

Assume $R1 = 0x2000$ before each instruction.

Instruction	Mode	Reads from	R1 afterwards
LDR R0, [R1]	register	0x2000	0x2000
LDR R0, [R1, #8]	offset	0x2008	0x2000
LDR R0, [R1, #8]!	pre-indexed	0x2008	0x2008
LDR R0, [R1], #8	post-indexed	0x2000	0x2008

The Writeback Indicator (!)

- The exclamation point ! explicitly commands auto-update (writeback) for pre-indexed access.
- Post-indexed access **implicitly updates** the base register without requiring a ! symbol.

1. Simple offset

```
LDR R0, [R1, #8]  
; address = R1 + 8  
; R1 unchanged
```

2. Pre-indexed: Calculate first, then access.

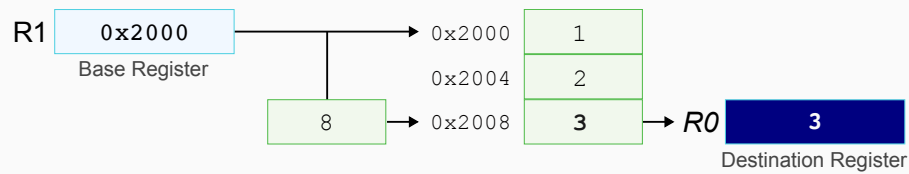
```
LDR R0, [R1, #8]!  
; address = R1 + 8  
; R1 = R1 + 8
```

3. Post-indexed: Access first, then calculate.

```
LDR R0, [R1], #8  
; address = R1  
; R1 = R1 + 8 (after)
```

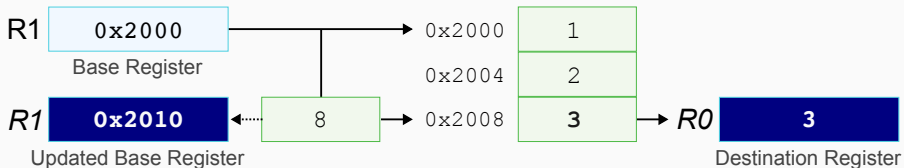
Read the brackets, not the mnemonic

- Offset **inside** the brackets → pre-indexed.
- Offset **outside** the brackets → post-indexed.
- Trailing ! → writeback.

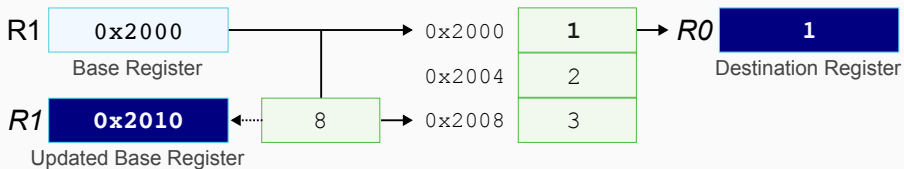


`R0` is loaded with the address pointed by `R1 + 8 = 0x2008`. `R1` is not updated.

Visualizing Pre-Indexed Access `STR R0, [R1, #16]!`



The address is computed *first*: $0x2000 + 16 = 0x2010$. This is both where R0 is loaded from, and where R1 ends up. One calculation, used twice.



R0 is loaded from address R1 already held — `0x2000` — *before* anything changes. R1 is updated *afterward*, to `0x2010`, ready for the next access.

Offset, pre-indexed, and post-indexed addressing are useful for any pointer, not only array indices:

- A **structure field** at a fixed offset from a base pointer — $[R1, \#4]$
- A **buffer** walked one element at a time, letting the pointer advance itself — $[R1], \#4$
- A **local variable in a stack frame** at a fixed offset from the frame pointer — $[R7, \#8]$

Every one of these uses a constant offset. No second register or shift is needed. The next section adds a register offset—and then a scaled register offset—when the offset itself is **data**, such as a run-time index.

- 1 Why This Matters
- 2 Arithmetic
- 3 Logic and Bit Manipulation
- 4 Shift Basics
- 5 The Barrel Shifter
- 6 Addressing Modes
- 7 Dynamic Indexed Modes**
- 8 Design Challenge

Form	Written	Typical C equivalent
Immediate offset	[R1, #4]	*(p + 1)
Register offset	[R1, R2]	*((char *)p + offset)
Scaled register offset	[R1, R2, LSL #2]	p[i]

The key idea

A register offset supplies a displacement at run time. A **scaled** register offset also multiplies that displacement by a power of two, which makes it ideal for array indexing.

For a 32-bit element:

$$\text{address} = \text{base} + (\text{index} \times 4)$$

Scaled Register Offset

```
LDR R0, [R1, R2, LSL #2]
```

- R1 — base address of the array
- R2 — the index
- LSL #2 — scale by 4, the element size

An immediate offset is fixed when you write the program; a register offset can change **while the program runs**, which is exactly what a loop counter needs.

Indexing without a multiply

The barrel shifter feeds directly into address generation, so scaling the index, adding the base, and loading the value all happen in **one instruction**. No separate shift or MUL is needed.

This is Chapter 3's formula, $\text{base} + (\text{index} * \text{element size})$, computed by hardware. It is why that multiplication never appears explicitly in compiler output.

Shift	Scales by, matching
LSL #1	2 — a 16-bit halfword (<code>int16_t</code>)
LSL #2	4 — a 32-bit word (<code>int32_t</code> , any pointer)

Array Indexing Becomes a Scaled Offset

C

```
int x = array[i];
```

For a 32-bit int, the compiler computes the address as $\text{array} + i \times 4$.

C type	Typical scaling	
int16_t *	LSL #1	(2 bytes)
int32_t *	LSL #2	(4 bytes)
uint32_t *	LSL #2	(4 bytes)

Typical ARM

```
; R1 = array
; R2 = i
LDR R0, [R1, R2, LSL #2]
```

Address calculation

$$R1 + (R2 \times 4)$$

The shift scales the index before it is added to the base address.

This is why array access on ARM is so cheap. If someone says “the barrel shifter is just a historical curiosity,” they have not looked at what the compiler produces for an array loop.

Array Indexing vs Pointer Arithmetic

Same address, different notation

```
int x = array[i];  
int x = *(array + i);  
  
int *p = array;  
int x = p[i];    // same as *(p + i)
```

In every case, i is an *element index*. Pointer arithmetic automatically accounts for the size of the pointed-to type.

What changes at the machine level?

The compiler turns the element index into a byte displacement:

$$i \times \text{sizeof}(\text{int})$$

A useful distinction

$p + i$ advances by i *elements*.

$(\text{char } *)p + i$ advances by i *bytes*.

Worked Example: Array Processing

C code

```
int array[10];
int sum = 0;
int i;

for (i = 0; i < 10; i++) {
    sum += array[i];
}
```

ARM assembly

```
LDR R1, =array      ; base address
MOV R0, #0          ; sum = 0
MOV R2, #0          ; i = 0

loop
LDR R3, [R1, R2, LSL #2]
ADD R0, R0, R3      ; sum += a[i]
ADD R2, R2, #1      ; i++
CMP R2, #10
BLT loop
```

This example combines

- LDR = to load an address
- a **scaled register offset** for the index
- register-only arithmetic
- CMP and a conditional branch for the loop

The LSL #2 handles the array arithmetic; no multiply instruction appears.

- AND masks, ORR sets, EOR toggles, BIC clears. **Read-modify-write** is how every bit field is changed without disturbing its neighbours.
- LSL/LSR/ASR/ROR shift on their own; the **flexible second operand** folds a shift into almost any instruction, at no extra cost.
- **Offset** leaves the base alone; **pre-indexed** $[Rn, \#n]!$ updates it and reads the new address; **post-indexed** $[Rn], \#n$ reads the old and always updates.
- A **scaled register offset** computes $\text{base} + \text{index} \times \text{size}$ in hardware, using the same **barrel shifter** — but only when the offset is itself data, as in array indexing.

Next: Chapter 7 — Control Flow, Functions and the Stack

You can now move and change data. Next, we decide what to do with it.

- 1 Why This Matters
- 2 Arithmetic
- 3 Logic and Bit Manipulation
- 4 Shift Basics
- 5 The Barrel Shifter
- 6 Addressing Modes
- 7 Dynamic Indexed Modes
- 8 Design Challenge

The brief

A colleague argues that the Cortex-M4 has a single-cycle hardware multiplier, so the barrel shifter is just a historical curiosity. They say the compiler should simply use MUL everywhere.

Judge this claim. Identify what a barrel-shifted operand gives you that MUL does not. Think about the instruction count, register pressure, and the fact that the shift happens on the path into the ALU, not as a separate step. Then find three constants for which multiplying with shifts is cheaper than MUL, and one constant for which MUL is clearly better. State what you would measure to settle this argument on real hardware.

Questions?

Introduction to ARM Cortex-M Microprocessors — Chapter 6